[MANUAL DE ESTUDOS]

Trilha de Estudos
Seu 1º Login Full-Stack

Do “não sei por onde começar” até o projeto funcionando

JavaScript · Node.js/Express · MySQL

IA entra só como ferramenta de consulta — quem constrói é você.

META A meta

Ao final você terá um sistema com **cadastro**, **login** e uma **página protegida**, feito por você, com senha criptografada, token de acesso e banco de dados. E, mais importante: vai **entender cada linha**, porque cada peça foi construída e testada separadamente antes de juntar.

O que você já tem

- Lógica de programação
- Rodar código no terminal
- Ler código simples de back-end
- HTML/CSS visual

O que falta (só 4 pontes)

- **HTTP:** como front e back conversam (pedido/resposta/JSON)
- **Assíncrono:** fetch e async/await
- **SQL** + ligar o Node ao MySQL
- **Segurança:** hash, JWT, cookie

! Por que parece que você “não sabe nada”?

Porque o projeto de login junta **6 assuntos de uma vez**. Ninguém aprende assim. A solução é estudar em **camadas**: cada fase termina num **mini-projeto que funciona**, e o login é só a soma das peças que você já dominou.

Legenda de cores (vale para o PDF todo)

Front-end / JS	Back-end / Express	Banco / SQL	Segurança	Integração

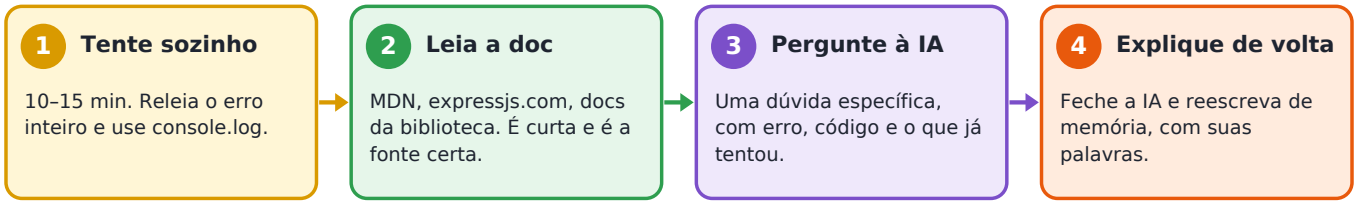
Leia primeiro: como usar esta trilha

CICLO Ciclo de cada sessão (1 a 2 horas)

10 min — responder as perguntas da fase anterior, sem olhar. **60–90 min** — estudar um bloco e **digitar** o código (nada de copiar e colar). **10 min** — escrever 3 linhas no "caderno de bordo":

• Hoje eu construí: ... • Quebrou quando: ... • Consertei com: ... • Ainda não entendo: ...

A regra da IA: os 3 passos

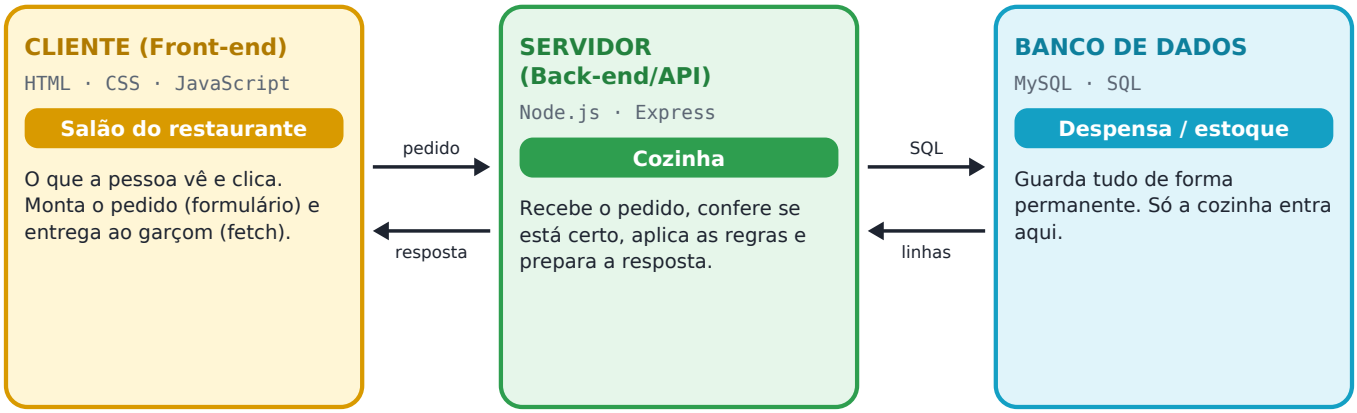


Evite	Prefira
"Faz um sistema de login pra mim"	"No Express 5 com mysql2/promise, como escrevo uma função que busca usuário por e-mail com prepared statement? Só a função, explicando linha a linha."
"Deu erro, arruma"	"Erro: [mensagem completa]. Código: [trecho]. Já tentei X e Y. Qual a causa provável e como confirmo?"
Colar o projeto todo e pedir para refazer	"Revisa só este middleware e aponta problemas de segurança, sem reescrever tudo."

✓ Teste do entendimento

Se você não consegue reescrever o trecho sem olhar, ainda não é seu. Volte, releia a doc e tente de novo antes de seguir para a próxima fase.

O mapa: restaurante + 7 fases



Analogia: o **garçom** é a API (HTTP + JSON): leva o pedido e traz a resposta. Você ainda nunca ligou salão e cozinha; essa ligação é o coração da trilha.

Roteiro das 7 fases

Fase	Você aprende	Mini-projeto (entrega)	Sessões
0	Ferramentas, terminal, Git	Hello World em Node + repositório	1
1	JavaScript essencial, JSON, async/await	5 desafios de funções e Promises	3
2	Formulário, DOM, fetch	Página que lê e envia dados a uma API pública	2
3	Node + Express, rotas, status HTTP	Lista de tarefas: front + API (dados em memória)	4
4	SQL, MySQL, pool, prepared statements	Mesma lista de tarefas, agora no banco	3
5	bcrypt, Zod, JWT, cookie HttpOnly, rate limit	API de auth testada no Thunder Client	4
6	Estrutura MVC e integração	Login completo: cadastro, login, página protegida	4

Sessão = 1 a 2 horas. Total: cerca de 21 sessões, ou 6 a 7 semanas a 3–4 sessões por semana. Ajuste ao seu calendário da faculdade — é melhor 3 sessões curtas por semana do que 1 maratona.

★ Regra de ouro de cada fase

Só avance quando o mini-projeto **roda** e você acerta as perguntas “sem olhar”. Pular fase é o que faz tudo parecer um monstro lá na frente.

? Por que não começar direto pelo login?

Porque o login usa **todas** as peças ao mesmo tempo (formulário, fetch, Express, SQL, hash, token). Se algo quebra, você não sabe de qual camada é o erro. Com mini-projetos, cada pedaço já funciona antes de entrar no sistema.

0 Preparar o terreno

~1 sessão

Instalar, abrir o terminal, criar o repositório

O que instalar

Ferramenta	Para quê	Confirme com
Node.js (versão LTS)	Rodar JavaScript fora do navegador (nosso back-end)	`node -v` e `npm -v`
VS Code	Editor. Instale a extensão **Thunder Client** (testar API)	Abrir uma pasta
Git	Histórico do projeto, cópia de segurança	`git --version`
MySQL Server + Workbench	Banco de dados e uma tela para rodar SQL	`mysql --version`

```
terminal

mkdir login-fullstack && cd login-fullstack
git init
echo "node_modules/" >> .gitignore
echo ".env" >> .gitignore          # segredos NUNCA vão para o Git
mkdir backend && cd backend
npm init -y                        # cria o package.json
```

```
primeiro teste

// backend/index.js  ->  rode com:  node index.js
console.log('Node funcionando!');
```

Checklist da fase 0

- [ ] Node, npm, Git e MySQL respondem a versão no terminal
- [ ] Consigo abrir o MySQL Workbench e conectar no servidor local
- [ ] Repositório criado, com **.gitignore** contendo node_modules e .env
- [ ] Primeiro commit feito (`git add . && git commit -m "inicio"`)

RESPONDA SEM OLHAR — Fase 0

1. Qual comando mostra a versão do Node instalada?
2. Para que serve o `.gitignore` e o que deve entrar nele neste projeto?
3. Qual a diferença entre `npm init -y` e `npm install express`?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

1 JavaScript que você realmente precisa

~3 sessões

Sem enrolar: só o que o projeto usa

Os 7 conceitos

Conceito	Onde vai aparecer no projeto
const / let, tipos, template string	Em todo lugar
Funções e arrow functions `() => {}`	Rotas, controllers, middlewares
Objetos e arrays (`find`, `map`, `filter`, `push`)	Lista de tarefas, resultados do banco
Desestruturação `const { email, senha } = req.body`	Pegar dados do pedido
JSON: `JSON.stringify` / `JSON.parse`	Corpo de toda requisição e resposta
Promise e **async/await**	fetch, banco, bcrypt: tudo é assíncrono
Módulos (`import`/`export`) e **try/catch**	Separar em arquivos. Use ESM: `"type": "module"` no package.json, sem misturar com require

☕ **Analogia: async/await**

Você pede uma pizza. A **Promise** é o comprovante: “vai chegar”. O **await** é você esperando *sem travar a casa inteira*. Quando chega, você segue. Banco, rede e senha demoram, então *toda* chamada deles precisa de await.

```
treine isto até ficar natural

const esperar = (ms) => new Promise((ok) => setTimeout(ok, ms));

async function buscarUsuario(id) {
  await esperar(500);                 // finge que foi ao banco
  return { id, nome: 'Rafael' };
}

async function main() {
  const usuario = await buscarUsuario(1);   // sem await, vem uma Promise
  console.log(usuario);
}
main();
```

MINI-PROJETO 1 — Cinco desafios

Entrega: um arquivo desafios.js que roda no terminal

- Função que recebe um array de usuários e devolve só os e-mails
- Função que busca um usuário por id num array (`find`) e devolve `undefined` se não achar
- Transformar objeto em texto JSON e voltar a objeto
- Reescrever o exemplo do garçom acima com `try/catch` e um erro proposital
- Duas funções async chamadas em sequência, depois com `Promise.all` em paralelo

Pronto quando: você explica a diferença entre sequência e paralelo sem olhar

RESPONDA SEM OLHAR — Fase 1

1. O que o `await` faz e onde ele pode ser usado?
2. O que `const { email, senha } = req.body` faz?
3. Qual a diferença entre `JSON.stringify` e `JSON.parse`?
4. Por que existem Promises? O que acontece se eu esquecer o await?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

2 Front-end: formulário, DOM e fetch

~2 sessões

A cara do sistema e o pedido que ela envia

O que aprender

- **HTML semântico:** `<form>`, `<label>`, `<input type="email">`, `type="password"`, `required`, `<button>`
- **DOM:** `querySelector`, `.value`, `addEventListener`, `textContent` (mostrar mensagens)
- **fetch:** GET e POST com JSON, ler a resposta, checar `resposta.ok`
- **DevTools (F12):** aba Console e aba **Network**. Você vai viver nelas.

```
frontend/js/login.js (modelo)

const form = document.querySelector('#form-login');

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const email = document.querySelector('#email').value;
  const senha = document.querySelector('#senha').value;

  const resposta = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, senha }),
  });
  const dados = await resposta.json();

  if (!resposta.ok) { mostrarErro(dados.erro); return; }
  window.location.href = '/dashboard.html';
});
```

Linha a linha

`addEventListener('submit', ...)`	Executa a função quando o formulário é enviado. Precisa ser **async** para usar await dentro.
`e.preventDefault()`	Impede o navegador de recarregar a página. Quem envia agora é o **fetch**.
`querySelector('#email').value`	Lê o que a pessoa digitou no campo de id *email*.
`await fetch(url, {...})`	Manda o pedido ao servidor e espera. Método POST, avisando que o corpo é JSON.
`JSON.stringify({ email, senha })`	Transforma o objeto em texto para viajar pela rede.

`resposta.ok`	Verdadeiro se o status for 200–299. **fetch não dá erro em 401 ou 400**: você precisa checar.

MINI-PROJETO 2 — Conversando com uma API pública

Entrega: uma página HTML que fala com a internet

- Pegar `https://jsonplaceholder.typicode.com/posts` com GET e listar os títulos na tela
- Criar um formulário que faz POST em `/posts` dessa mesma API (ela finge salvar) e mostra a resposta
- Abrir o **Network** e identificar: método, status, payload enviado e resposta recebida

Pronto quando: você aponta cada informação na aba Network sem procurar

RESPONDA SEM OLHAR — Fase 2

1. Por que usar `e.preventDefault()` no submit?
2. O fetch lança erro quando o servidor responde 401? Como você detecta o problema?
3. Qual cabeçalho avisa o servidor de que o corpo é JSON?
4. Qual linha lê o valor digitado no campo de id email?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

3 Back-end: Node + Express

~4 sessões

A cozinha: receber pedidos e responder

Antes do código: o básico de HTTP

Pedaço	O que é	Exemplo
Método	A intenção do pedido	GET ler · POST criar · PUT atualizar · DELETE apagar
URL / rota	O endereço do que você quer	`/api/auth/login`
Corpo (body)	Os dados enviados, em JSON	`{ "email": "...", "senha": "..." }`
Status	O resultado, em número	**200** ok · **201** criado · **400** dados ruins · **401** não autenticado · **403** sem permissão · **404** não achou · **409** conflito · **500** erro nosso

Seu primeiro servidor

backend/server.js (modelo)

```
// backend/server.js     (npm i express)
import express from 'express';
import path from 'node:path';

const app = express();
app.use(express.json());
app.use(express.static(path.join(import.meta.dirname, '../frontend')));

const tarefas = [];                          // "banco" provisorio

app.get('/api/tarefas', (req, res) => {
  res.json(tarefas);
});

app.post('/api/tarefas', (req, res) => {
  const { titulo } = req.body;
  if (!titulo) return res.status(400).json({ erro: 'titulo obrigatorio' });
  const nova = { id: tarefas.length + 1, titulo };
  tarefas.push(nova);
  res.status(201).json(nova);
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

Linha a linha

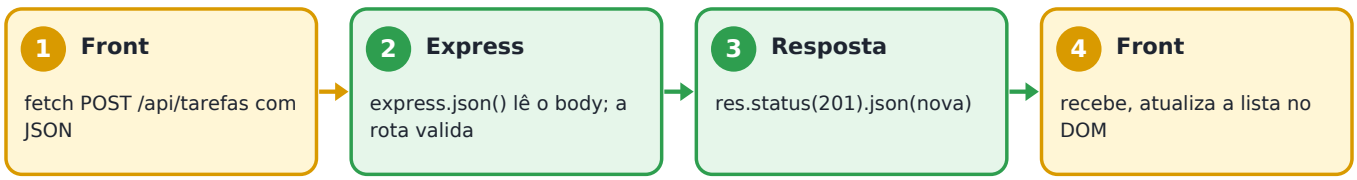
`app.use(express.json())`	Middleware que lê o corpo JSON e coloca em `req.body`. Sem ele, `req.body` vem vazio.
`app.use(express.static(...))`	Entrega os arquivos do front (HTML/CSS/JS). Front e back ficam na **mesma origem**, então sem dor de cabeça com CORS.
`app.get('/api/tarefas', (req, res) => ...)`	**Rota:** quando chegar um GET nesse endereço, rode esta função. `req` = pedido, `res` = resposta.
`res.status(400).json({...})`	Define o status e devolve JSON. O `return` evita continuar executando.
`res.status(201).json(nova)`	201 = criado. O front recebe o objeto novo.
`app.listen(3000, ...)`	Liga o servidor na porta 3000.

ANALOGIA Middleware = fila de seguranças

Todo pedido passa por uma fila de funções antes de chegar na rota. Cada uma decide: `next()` (“pode seguir”) ou responde e barra. É assim que validação e login funcionam depois.

! CORS em uma frase

Se o front abre em uma porta (ex.: 5500) e a API em outra (3000), o navegador bloqueia por segurança. Começando com o Express servindo o front, isso nem aparece. Aprenda o pacote `cors` depois.



MINI-PROJETO 3 — Lista de tarefas (o “amadureceu” da trilha)

Entrega: front + API conversando, dados em memória

- Testar a API sozinha no **Thunder Client** (GET e POST) antes de criar o front
- Criar página com formulário e lista; ao abrir, `GET` preenche; ao enviar, `POST` e atualiza
- Adicionar DELETE por id e tratar 404
- Reiniciar o servidor e perceber que os dados somem (motivo da fase 4)

Pronto quando: você desenha de cabeça o caminho de um POST do clique até a tela atualizar

RESPONDA SEM OLHAR — Fase 3

1. Qual a diferença entre rota e controller?
2. O que o `express.json()` faz?
3. Que status você usa para: criado, dados inválidos, não autenticado, sem permissão, e-mail repetido?
4. O que é CORS e como evitamos no começo?
5. O que o `next()` faz num middleware?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

4 Banco de dados: SQL e MySQL

~3 sessões

A despensa: guardar para sempre

SQL em 6 comandos

```
rode no MySQL Workbench primeiro, na mão

CREATE DATABASE login_app CHARACTER SET utf8mb4;
USE login_app;

CREATE TABLE usuarios (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  nome       VARCHAR(100) NOT NULL,
  email      VARCHAR(150) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL,
  papel      ENUM('USUARIO','ADMIN') NOT NULL DEFAULT 'USUARIO',
  criado_em  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO usuarios (nome, email, senha_hash) VALUES ('Teste', 'a@a.com', 'x');
SELECT * FROM usuarios WHERE email = 'a@a.com';
UPDATE usuarios SET nome = 'Novo' WHERE id = 1;
DELETE FROM usuarios WHERE id = 1;
```

LEITURA Em português claro

PRIMARY KEY = o CPF da linha (único). **UNIQUE** no e-mail = o banco recusa dois cadastros iguais. **NOT NULL** = campo obrigatório. **senha_hash** guarda o hash, nunca a senha (fase 5).

Ligando o Node ao MySQL

modelo: db.js + um model

```
// backend/db.js           (npm i mysql2 dotenv)
import 'dotenv/config';
import mysql from 'mysql2/promise';

export const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  connectionLimit: 10,
});

// backend/models/usuario.model.js
export async function buscarPorEmail(email) {
  const [linhas] = await pool.execute('SELECT * FROM usuarios WHERE email = ?', [email]);
  return linhas[0];                // undefined se não achou
}

export async function criarUsuario(nome, email, senhaHash) {
  const [r] = await pool.execute(
    'INSERT INTO usuarios (nome, email, senha_hash) VALUES (?, ?, ?)',
    [nome, email, senhaHash]);
  return r.insertId;
}
```

ANALOGIAS Os dois conceitos que importam

Pool de conexões = caixas de supermercado: várias abertas e reaproveitadas, em vez de abrir uma loja nova a cada cliente.

? (prepared statement) = envelope lacrado: o que vem de fora entra como *dado*, nunca como *ordem*. Isso barra **SQL Injection**. Nunca monte SQL com `' + email + '`.

SEGREDOS Arquivo .env

Segredos ficam em `backend/.env` (fora do Git):

`PORT=3000 · DB_HOST=localhost · DB_USER=root · DB_PASSWORD=... · DB_NAME=login_app · JWT_SECRET=...`

MINI-PROJETO 4 — A lista de tarefas ganha banco

Entrega: a mesma lista da fase 3, mas persistente

- Criar a tabela `tarefas` (id, titulo, criada_em)
- Criar `db.js` e testar com `SELECT 1`
- Trocar o array por funções de model: `listar()`, `criar(titulo)`, `apagar(id)`
- Reiniciar o servidor e ver que os dados **continuam lá**

Pronto quando: os dados sobrevivem ao reinicio e não há SQL fora dos models

RESPONDA SEM OLHAR — Fase 4

1. Por que usar `WHERE email = ?` com `[email]` em vez de concatenar?
2. O que é um pool de conexões?
3. Para que servem PRIMARY KEY e UNIQUE?
4. Em qual camada do projeto o SQL deve ficar?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

5 Segurança do login

~4 sessões

Hash, validação, token, cookie e proteção de força bruta

	Autenticação	Autorização
Pergunta	**Quem é você?**	**O que você pode fazer?**
Analogia	Crachaá na portaria do prédio	Quais andares o crachaá abre
No projeto	e-mail + senha, depois token	papel USUARIO vs ADMIN
Erro HTTP	**401**	**403**

5.1 Senha: bcrypt (hash)

☕ **Analogia: liquidificador**

Você transforma fruta em vitamina, mas não consegue transformar a vitamina em fruta. No login, você bate a senha digitada do mesmo jeito e **compara as vitaminas**. Se o banco vazar, ninguém lê as senhas.

```
as duas únicas linhas que você precisa

import bcrypt from 'bcrypt';                         // npm i bcrypt
const hash = await bcrypt.hash(senha, 12);          // cadastro: salva ISTO no banco
const ok   = await bcrypt.compare(senha, hashSalvo); // login: true ou false
```

O número **12** (cost) deixa o cálculo propositalmente lento, o que atrapalha quem tenta adivinhar bilhões de senhas. O **salt** (tempero aleatório) já vem embutido no hash, então duas senhas iguais geram hashes diferentes.

5.2 Validação com Zod (catraca 1)

```
backend/middlewares/validar.js (confira a doc da versão instalada)

import { z } from 'zod';                             // npm i zod

export const cadastroSchema = z.object({
  nome:  z.string().min(2),
  email: z.string().email(),
  senha: z.string().min(12),
});

export const validar = (schema) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) return res.status(400).json({ erro: 'Dados inválidos' });
  req.body = r.data;
  next();                                            // passou na catraca
};
```

5.3 Token: JWT (a pulseira do evento)

☕ **Analogia**

Depois de provar quem você é na portaria, ganha uma **pulseira**. Nas próximas portas basta mostrá-la. O JWT tem 3 partes: **Header** (algoritmo), **Payload** (id e papel) e **Signature** (assinatura feita com o `JWT_SECRET`).

! Cuidado

O payload é apenas **codificado**, não secreto. Qualquer um lê. **Nunca** pôr senha ali. A assinatura só garante que ninguém *alterou* o token.

```
no controller de login (modelo)

import jwt from 'jsonwebtoken';                      // npm i jsonwebtoken cookie-parser

const token = jwt.sign({ id: usuario.id, papel: usuario.papel },
                       process.env.JWT_SECRET, { expiresIn: '15m' });

res.cookie('token', token, {
  httpOnly: true,                  // JavaScript do navegador não lê (barra XSS)
  sameSite: 'strict',              // não vai em requisições de outros sites (barra CSRF)
  secure: process.env.NODE_ENV === 'production',   // só HTTPS em produção
  maxAge: 15 * 60 * 1000,
});
```

5.4 Guardião da rota protegida

```
backend/middlewares/autenticar.js (modelo)

// app.use(cookieParser()) no server.js, antes das rotas
export function autenticar(req, res, next) {
  const token = req.cookies.token;
  if (!token) return res.status(401).json({ erro: 'Não autenticado' });
  try {
    req.usuario = jwt.verify(token, process.env.JWT_SECRET);   // { id, papel, exp }
    next();
  } catch {
    res.status(401).json({ erro: 'Token inválido ou expirado' });
  }
}

export const exigirPapel = (papel) => (req, res, next) =>
  req.usuario.papel === papel ? next() : res.status(403).json({ erro: 'Sem permissão' });
```

5.5 Freio de força bruta (catraca 2)

```
bloqueia o mesmo IP após 10 tentativas em 15 min (vira 429)

import rateLimit from 'express-rate-limit';          // npm i express-rate-limit
export const limitaLogin = rateLimit({ windowMs: 15 * 60 * 1000, limit: 10 });
// uso:  router.post('/login', limitaLogin, validar(loginSchema), login);
```

★ Boas práticas que quase todo tutorial esquece

- Mensagem de login **genérica**: “E-mail ou senha inválidos”. Não diga qual dos dois errou.
- E-mail repetido no cadastro: **409**. Nunca devolva `senha_hash` ao front.
- Logout = `res.clearCookie('token')`. Valide sempre no back, mesmo que o HTML já valide.
- Nunca devolva o erro bruto do SQL ao cliente: registre no servidor e responda algo genérico.

MINI-PROJETO 5 — API de autenticação (só back, testada no Thunder Client)

Entrega: rotas funcionando sem nenhuma tela

- `POST /api/auth/cadastro` → 201; repetindo o e-mail → 409; senha curta → 400
- `POST /api/auth/login` → 200 com cookie; senha errada → 401
- `GET /api/perfil` com cookie → 200; sem cookie → 401
- Conferir no Workbench que a coluna guarda o hash, nunca a senha

Pronto quando: todos os status acima aparecem como esperado

RESPONDA SEM OLHAR — Fase 5

1. Por que não salvar a senha em texto puro? E como o login confere se o hash é irreversível?
2. O que tem no payload do JWT e por que não pôr a senha ali?
3. O que o cookie HttpOnly protege? E o SameSite?
4. Qual a diferença entre autenticação e autorização? E entre 401 e 403?
5. Por que a mensagem de erro do login deve ser genérica?

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

6 Juntar tudo: o projeto final

~4 sessões

Estrutura MVC, fluxos e integração com o front

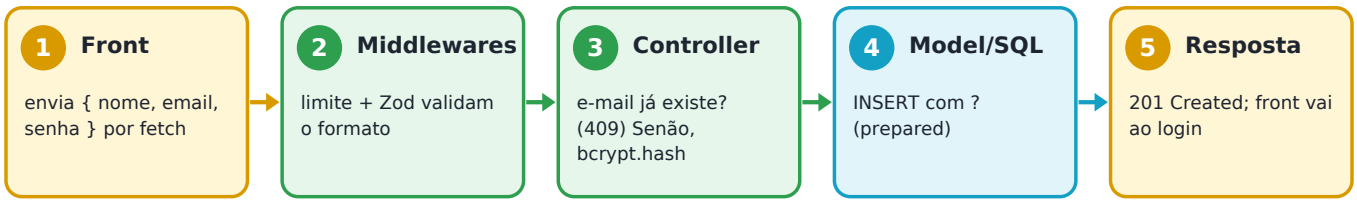
A estrutura de pastas

```
tudo o que você já praticou, cada coisa no seu lugar

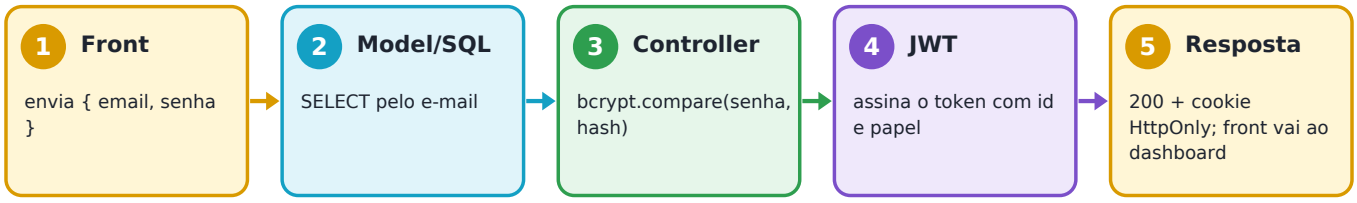
login-fullstack/
├── .gitignore                          <- node_modules/ e .env
├── frontend/
│   ├── cadastro.html  login.html  dashboard.html
│   ├── css/estilo.css
│   └── js/  cadastro.js  login.js  dashboard.js
└── backend/
    ├── .env                            <- segredos (NÃO vai ao Git)
    ├── package.json
    ├── server.js                       <- cria o app e liga as rotas
    ├── db.js                           <- pool do MySQL
    ├── routes/auth.routes.js           <- mapa: URL + método
    ├── controllers/auth.controller.js  <- a lógica (cérebro)
    ├── models/usuario.model.js         <- único lugar com SQL
    └── middlewares/  validar.js  autenticar.js  limitaLogin.js
```

Os dois fluxos

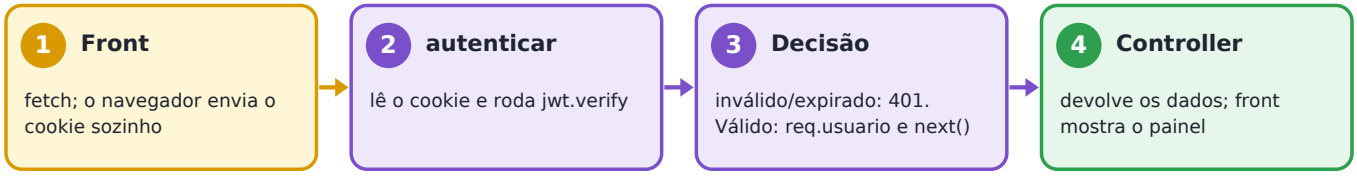
Cadastro (POST /api/auth/cadastro)



Login (POST /api/auth/login)



Rota protegida (GET /api/perfil)



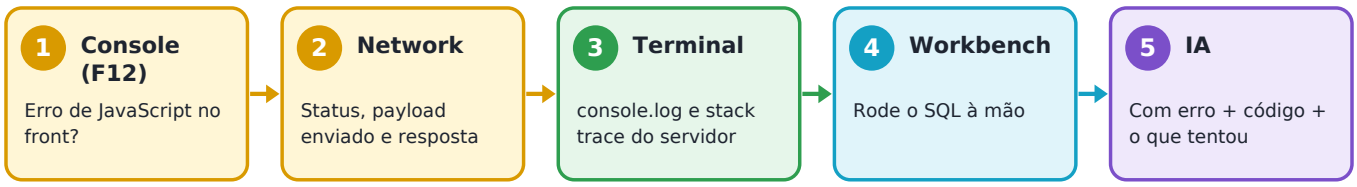
Ordem de construção (commit a cada passo)

- [ ] **Pastas + dependências:** `npm i express mysql2 dotenv bcrypt jsonwebtoken zod cookie-parser express-rate-limit` e no package.json: `"type": "module"` e script `"dev": "node --watch server.js"`
- [ ] **.env e .gitignore** conferidos antes do 1º commit
- [ ] **Banco:** CREATE DATABASE e CREATE TABLE usuarios (fase 4)
- [ ] **db.js** e teste de conexão com SELECT 1
- [ ] **usuario.model.js:** buscarPorEmail e criarUsuario
- [ ] **validar.js** + schemas (cadastro e login)
- [ ] **Controller de cadastro** (409, hash, insert, 201) e teste no Thunder Client
- [ ] **Controller de login** (compare, JWT, cookie) e teste
- [ ] **autenticar.js** + rota `GET /api/perfil` e teste (com e sem cookie)
- [ ] **limitaLogin** aplicado na rota de login
- [ ] **Front:** cadastro.html/js, depois login.html/js, depois dashboard.html/js (que chama /api/perfil e, se 401, volta ao login)
- [ ] **Logout:** rota que limpa o cookie + botão no dashboard

Pronto quando (testes manuais)

- [ ] Cadastro com e-mail repetido devolve **409**; senha curta devolve **400**
- [ ] Login com senha errada devolve **401** e mensagem genérica
- [ ] **/api/perfil** sem cookie devolve 401; com cookie devolve seus dados
- [ ] No DevTools (Application → Cookies) o token aparece com **HttpOnly** marcado
- [ ] Repetir login errado rápido até aparecer **429**
- [ ] No banco, a coluna senha_hash tem o hash; **.env não está no Git** (confira com `git status`)

Quando travar: onde está o bug?



+ Depois que o básico estiver sólido (extras)

- Rotas só para ADMIN com `exigirPapel`, pacote `helmet`, `cors` para front separado
- Refresh token, recuperação de senha por e-mail, deploy com HTTPS
- Testes automatizados, ORM (Prisma) e trocar o front por React

RESPONDA SEM OLHAR — Fase 6

1. Em que ordem as camadas são percorridas num cadastro, do clique até a resposta?
2. Apareceu um erro no front: por onde você começa a investigar?
3. Cite três coisas que nunca podem ir para o Git.

Responda em voz alta ou no caderno. Depois confira no gabarito (final do PDF).

Glossário rápido

Termo	Em português claro
API	O garçom: conjunto de rotas pelas quais o front pede coisas ao back
Endpoint / rota	O endereço + método de um pedido, como `POST /api/auth/login`
Request / Response	Pedido enviado / resposta recebida
JSON	Texto no formato de objeto JavaScript usado para trocar dados
Status code	Número que resume o resultado (200, 401, 404...)
Middleware	Função que fica no meio do caminho (segurança da fila)
Controller / Model	Cérebro da lógica / mensageiro do banco (onde vive o SQL)
Prepared statement	Consulta com ? em que o valor entra só como dado, nunca como ordem
Connection pool	Conexões com o banco reaproveitadas (caixas do mercado)
Hash / salt	Vitamina sem volta / tempero aleatório embutido
JWT	Pulseira digital assinada com id e papel (legível, mas à prova de adulteração)
Cookie HttpOnly	Cookie que o JavaScript da página não consegue ler
XSS	Script malicioso injetado na página que tenta roubar dados
CSRF	Outro site tenta usar seu cookie para agir em seu nome (SameSite ajuda)
SQL Injection	Atacante coloca comandos SQL no input. Prepared statement resolve
CORS	Regra do navegador sobre chamadas entre origens diferentes
.env	Arquivo de segredos, nunca versionado

Onde estudar (documentação oficial)

- **developer.mozilla.org** (MDN): HTML, DOM, fetch, JavaScript
- **expressjs.com** e **nodejs.org**: Express e Node
- **dev.mysql.com/doc**: SQL e MySQL
- **zod.dev**, e as páginas no npm/GitHub de bcrypt, jsonwebtoken, mysql2 e express-rate-limit

As bibliotecas mudam de versão. Sempre confira a doc da versão que você instalou.

Gabarito das perguntas

Fase 0

1. **node -v**.
2. Diz ao Git o que não versionar: **node_modules/** e **.env** (segredos).
3. **init** cria o package.json; **install** baixa a biblioteca, grava em node_modules e registra em dependencies.

Fase 1

1. Pausa a função até a Promise resolver, sem travar o resto. Só em função **async** (ou topo de módulo ESM).
2. Desestruturação: cria as variáveis email e senha a partir das propriedades de req.body.
3. stringify: objeto vira texto; parse: texto vira objeto.
4. Promise representa um valor que ainda vai chegar. Sem await, você recebe a Promise, não o valor.

Fase 2

1. Evita recarregar a página; quem envia passa a ser o fetch.
2. Não. Só rejeita em falha de rede. Verifique **resposta.ok** ou o status.
3. **Content-Type: application/json**.
4. document.querySelector('#email').value

Fase 3

1. Rota liga URL + método a uma função; controller contém a lógica.
2. Lê o corpo JSON e coloca em req.body.
3. 201, 400, 401, 403, 409.
4. O navegador bloqueia chamadas entre origens sem permissão do servidor. Começamos servindo o front pelo Express (mesma origem).
5. Passa para o próximo middleware ou rota.

Fase 4

1. O valor vai como dado, não como código: impede SQL Injection.
2. Um conjunto de conexões reaproveitadas, em vez de abrir uma por requisição.
3. PK identifica cada linha de forma única; UNIQUE impede valores repetidos (e-mail).
4. Nos **models**.

Fase 5

1. Se o banco vazar, as senhas vazam. Como o hash não volta, usa-se **bcrypt.compare(senhaDigitada, hashSalvo)**.
2. Id e papel. O payload é apenas codificado: qualquer um lê. A assinatura só garante que não foi alterado.
3. HttpOnly impede o JavaScript (XSS) de ler o cookie; SameSite reduz CSRF.
4. Quem é você vs o que pode fazer. 401 = não autenticado; 403 = autenticado, sem permissão.
5. Para não revelar se o e-mail existe.

Fase 6

1. Rota → middlewares (limite e Zod) → controller → model → MySQL → resposta 201.
2. DevTools: **Console** e **Network** (status, payload, resposta); depois o terminal do servidor; depois o SQL à mão no Workbench; só então a IA, com contexto.
3. **.env**, **node_modules/** e qualquer arquivo com chaves ou dados reais.

> **★ Você consegue**
>
> Ninguém nasce sabendo ligar front, back e banco: são 4 pontes pequenas, atravessadas uma por vez. Abra o editor, comece pela fase 0 e só passe para a próxima quando o mini-projeto rodar.